



# Robotics Interest Group



## ROUND 1 INDUCTION REPORT

|                               |               |
|-------------------------------|---------------|
| Name:                         | SHIVADHARAN S |
| Roll No:                      | ██████████    |
| Phone No:                     | ██████████    |
| Link to Google drive/project: | ██████████    |
| Remarks:                      |               |

### Your Problem Statement:

write python code to make a text classifier where the output is to convey if the text is a positive or negative statement, download and use pre-existing AI Text-Classifier models from online sources like media pipe solutions for text

classification, the output should be a percentage of positivity and negativity in the statement.

## **Solution:**

To create the text classifier I developed five different models each using a different approach with increasing complexity

For the final model I created my **own** NLM which was trained using Twitter sentiment data that I found online

I'll start the walkthrough from the simplest and most intuitive approach and then gradually move toward more advanced models

### **1. Rule-Based Model (Basic Approach)**

To understand whether a statement is positive or negative, I first:

- Defined a few terms considered positive
- Defined a few terms considered negative
- Used simple if-statements to generate the sentiment of the given statement

This approach is very intuitive and easy to understand

## SOURCE CODE:

```
thor_mark1.py > ...
1  #rule based approach(no use of nlms)
2  positive_words = {
3      "good", "great", "excellent", "happy", "love", "amazing",
4      "awesome", "fantastic", "nice", "wonderful"
5  }
6
7  negative_words = {
8      "bad", "terrible", "awful", "sad", "hate", "poor",
9      "worst", "horrible", "disappointing", "angry"
10 }
11
12
13 def predict(text):
14     words = text.lower().split()
15
16     positive_count = 0
17     negative_count = 0
18
19     for word in words:
20         if word in positive_words:
21             positive_count += 1
22         elif word in negative_words:
23             negative_count += 1
24
25     if positive_count > negative_count:
26         sentiment = "POSITIVE"
27     elif negative_count > positive_count:
28         sentiment = "NEGATIVE"
29     else:
30         sentiment = "NEUTRAL"
31     total = positive_count + negative_count
32     if total > 0:
33         positive_percent = (positive_count / total) * 100
34         negative_percent = (negative_count / total) * 100
35     else:
36         positive_percent = negative_percent = 0
37
38
39     return sentiment, positive_count, negative_count , positive_percent , negative_percent
40
41
42
43 text = "Absolutely terrible experience, nothing worked and I regret buying it 🤬💀"
44 sentiment, pos, neg ,pospercent , negpercent= predict(text)
45
46 print("SENTIMENT: ", sentiment)
47 print("POSITIVE WORD COUNT: ", pos)
48 print("NEGATIVE WORD COUNT: ", neg)
49 print("POSITIVE SCORE: " , pospercent)
50 print("NEGATIVE SCORE: ",negpercent)
```

## OUTPUT:

```
• SENTIMENT:  NEGATIVE
POSITIVE WORD COUNT:  0
NEGATIVE WORD COUNT:  1
POSITIVE SCORE:  0.0
NEGATIVE SCORE:  100.0
```

**Limitation:**

The problem with this approach is that:

- Every positive or negative word is given the same weight
- In reality words do not contribute equally to sentiment

**2. Naïve Bayes Approach**

The next approach I took was using the Naïve Bayes theorem to calculate probabilities

This approach is:

- More mathematical
- More proper than simple if-statements

To implement this, I had to learn about a few more models and concepts.

## SOURCE CODE:

```
1  #implementing machine learning and naive bayes
2  from sklearn.pipeline import Pipeline
3  from sklearn.feature_extraction.text import TfidfVectorizer
4  from sklearn.naive_bayes import MultinomialNB
5
6  #training data
7  texts = [
8      "I love this movie",
9      "This product is amazing",
10     "Very happy with the service",
11     "I hate this item",
12     "This is the worst experience",
13     "Terrible quality and bad support"
14 ]
15
16 labels = [
17     "positive",
18     "positive",
19     "positive",
20     "negative",
21     "negative",
22     "negative"
23 ]
24
25 model = Pipeline([("tfidf",TfidfVectorizer()),("nb",MultinomialNB())])
26
27 model.fit(texts,labels)
28
29 text = ["i would love to join rig"]
30 #predict prb must me a list
31 probabilty = model.predict_proba(text)[0]
32 order = model.classes_
33
34 print(order[0],"=", round(probabilty[0]*100 , 2))
35 print(order[1],"=", round(probabilty[1]*100 , 2))
36
```

## OUTPUT:

```
negative = 37.49
positive = 62.51
```

## Limitation:

- It still considers all words equally
- The need to create weights for each word becomes important

## 3. Logistic Regression (Machine Learning Approach)

In the next approach, I used Logistic Regression, which is a machine learning algorithm.

This resulted in:

- More professional and accurate results

- A model that learns patterns instead of relying on fixed rules

#### SOURCE CODE:

```
1  #implementing machine learning using logistic regression
2  from sklearn.pipeline import Pipeline
3  from sklearn.feature_extraction.text import TfidfVectorizer
4  from sklearn.linear_model import LogisticRegression
5  #training data
6  texts = [
7      "I love this movie",
8      "This product is amazing",
9      "Very happy with the service",
10     "I hate this item",
11     "This is the worst experience",
12     "Terrible quality and bad support"
13 ]
14
15 labels = [
16     "positive",
17     "positive",
18     "positive",
19     "negative",
20     "negative",
21     "negative"
22 ]
23 model = Pipeline([
24     ("tfidf", TfidfVectorizer()),
25     ("lr", LogisticRegression(max_iter=1000))
26 ])
27
28 model.fit(texts, labels)
29
30 probability = model.predict_proba(["rig is challenging but i love it"])[0]
31 order = model.classes_
32
33 print(order[0], "=", round(probability[0]*100, 2))
34 print(order[1], "=", round(probability[1]*100, 2))
35
```

#### OUTPUT:

```
negative = 44.61
positive = 55.39
```

#### Challenge:

Since this is machine learning, it requires a large amount of data.

So I trained the model using:

- A custom-made dataset (added in Google Drive)
- Twitter sentiment data

## SOURCE CODE:

```
1  #making thor to learn from real life data
2  import pandas as pd
3  import emoji
4  import re
5  import ftfy
6  from sklearn.pipeline import Pipeline
7  from sklearn.feature_extraction.text import TfidfVectorizer
8  from sklearn.linear_model import LogisticRegression
9  from sklearn.metrics import classification_report
10
11 def clean_text(text):
12     text = str(text)
13     text = ftfy.fix_text(text)
14     text = emoji.demojize(text)
15     text = re.sub(r"http\S+|www\S+", "", text)
16     text = re.sub(r"@w+", "", text)
17     text = re.sub(r"[^a-zA-Z_]", " ", text)
18     return text.lower().strip()
19
20
21
22
23 #training data
24 df = pd.read_csv("twitter_trainingclean2.csv")
25 df["text"] = df["text"].apply(clean_text)
26 val_df = pd.read_csv("twitter_validationclean2.csv")
27 val_df["text"] = val_df["text"].apply(clean_text)
28
29 df = df[df["labels"] != "Irrelevant"]
30 val_df = val_df[val_df["labels"] != "Irrelevant"]
```

```
31
32
33 texts = df["text"].tolist()
34 labels = df["labels"].tolist()
35
36 texts_val = val_df["text"].tolist()
37 labels_val = val_df["labels"].tolist()
38
39 model = Pipeline([
40     ("tfidf", TfidfVectorizer()),
41     ("lr", LogisticRegression(max_iter=1000))
42 ])
43
44 model.fit(texts, labels)
45 val_preds = model.predict(texts_val)
46 report = classification_report(labels_val, val_preds)
47 print(report)
48
49 from sklearn.metrics import accuracy_score
50
51 accuracy = accuracy_score(labels_val, val_preds)
52 print(f"Accuracy: {accuracy * 100:.2f}%")
53
54
55 probabilty = model.predict_proba(["i am very sad"])[0]
56 order = model.classes_
57
58 print(order[0], "=", round(probabilty[0]*100, 2))
59 print(order[1], "=", round(probabilty[1]*100, 2))
60 print(order[2], "=", round(probabilty[2]*100, 2))
```

## OUTPUT:

|                  | precision | recall | f1-score | support |
|------------------|-----------|--------|----------|---------|
| Negative         | 0.91      | 0.95   | 0.93     | 266     |
| Neutral          | 0.96      | 0.90   | 0.93     | 285     |
| Positive         | 0.92      | 0.94   | 0.93     | 277     |
| accuracy         |           |        | 0.93     | 828     |
| macro avg        | 0.93      | 0.93   | 0.93     | 828     |
| weighted avg     | 0.93      | 0.93   | 0.93     | 828     |
| Accuracy: 92.63% |           |        |          |         |
| Negative = 72.62 |           |        |          |         |
| Neutral = 15.28  |           |        |          |         |
| Positive = 12.1  |           |        |          |         |

## 4. Final Model and Improvements

This was a big breakthrough in analyzing text sentiment

However, the model can still be improved by:

- Training it with data containing sarcasm
- Including idioms and contextual expressions

To enhance usability, I also:

- Added a GUI to beautify and simplify interaction

## SOURCE

## CODE:

```
1  # thor_model.py
2  import pandas as pd
3  import emoji
4  import re
5  import ftfy
6  from sklearn.pipeline import Pipeline
7  from sklearn.feature_extraction.text import TfidfVectorizer
8  from sklearn.linear_model import LogisticRegression
9  import joblib
10 import os
11
12
13 def clean_text(text):
14     text = str(text)
15     text = ftfy.fix_text(text)
16     text = emoji.demojize(text)
17     text = re.sub(r"http\S+|www\S+", "", text)
18     text = re.sub(r"@w+", "", text)
19     text = re.sub(r"^[a-zA-Z_ ]", " ", text)
20     return text.lower().strip()
21
22
23 def train_model():
24     df = pd.read_csv("twitter_trainingclean2.csv")
25     df["text"] = df["text"].apply(clean_text)
26     df = df[df["labels"] != "Irrelevant"]
27
```

```
28     texts = df["text"].tolist()
29     labels = df["labels"].tolist()
30
31     model = Pipeline([
32         ("tfidf", TfidfVectorizer()),
33         ("lr", LogisticRegression(max_iter=1000))
34     ])
35
36     model.fit(texts, labels)
37
38     # Save model
39     joblib.dump(model, "thor_sentiment_model.pkl")
40     return model
41
42 def load_model():
43     if os.path.exists("thor_sentiment_model.pkl"):
44         model = joblib.load("thor_sentiment_model.pkl")
45         return model
46     else:
47         return train_model()
48
49 def predict_sentiment(text, model):
50     cleaned = clean_text(text)
51     probs = model.predict_proba([cleaned])[0]
```

```
52     classes = model.classes_
53     predicted = classes[probs.argmax()]
54     return predicted, dict(zip(classes, probs))
55
56
57 load_model()
```

```

1  # thor_ui.py
2  import tkinter as tk
3  from thor_model import load_model, predict_sentiment
4  import math
5
6  model = load_model()
7
8
9  def analyze_text():
10     user_text = input_box.get("1.0", tk.END).strip()
11     if not user_text:
12         result_label.config(text="enter text ")
13         return
14
15     predicted, probs_dict = predict_sentiment(user_text, model)
16
17     result = f"Prediction: {predicted}\n\n"
18     for cls, prob in probs_dict.items():
19         result += f"{cls}: {prob*100:.2f}%\n"
20
21     result_label.config(text=result)
22
23
24 root = tk.Tk()
25 root.title("THOR")
26 root.geometry("520x450")
27 root.configure(bg="#05070d")

```

```

28 root.resizable(False, False)
29
30 canvas = tk.Canvas(
31     root,
32     width=520,
33     height=120,
34     bg="#05070d",
35     highlightthickness=0
36 )
37 canvas.pack()
38
39
40 hammer_head = canvas.create_rectangle(230, 40, 290, 70, fill="#8ab4ff", outline="")
41 hammer_handle = canvas.create_rectangle(258, 70, 262, 105, fill="#c7d2fe", outline="")
42 glow = canvas.create_oval(215, 25, 305, 115, outline="#2563eb", width=2)
43
44
45 angle = 0
46
47 def animate_hammer():
48     global angle
49     angle += 0.08
50
51     float_y = math.sin(angle) * 4
52     glow_alpha = abs(math.sin(angle))
53
54     canvas.move(hammer_head, 0, float_y)

```

```

55     canvas.move(hammer_handle, 0, float_y)
56     canvas.move(glow, 0, float_y)
57
58     canvas.itemconfig(
59         glow,
60         outline=f"#2563eb"
61     )
62
63     root.after(40, animate_hammer)
64
65     animate_hammer()
66
67     title = tk.Label(
68         root,
69         text="⚡ THOR ⚡",
70         font=("Segoe UI", 20, "bold"),
71         bg="█"#05070d",
72         fg="█"#93c5fd"
73     )
74     title.pack(pady=(5, 0))
75
76     subtitle = tk.Label(
77         root,
78         text="Text-based Human Opinion Reader",
79         font=("Segoe UI", 10),

```

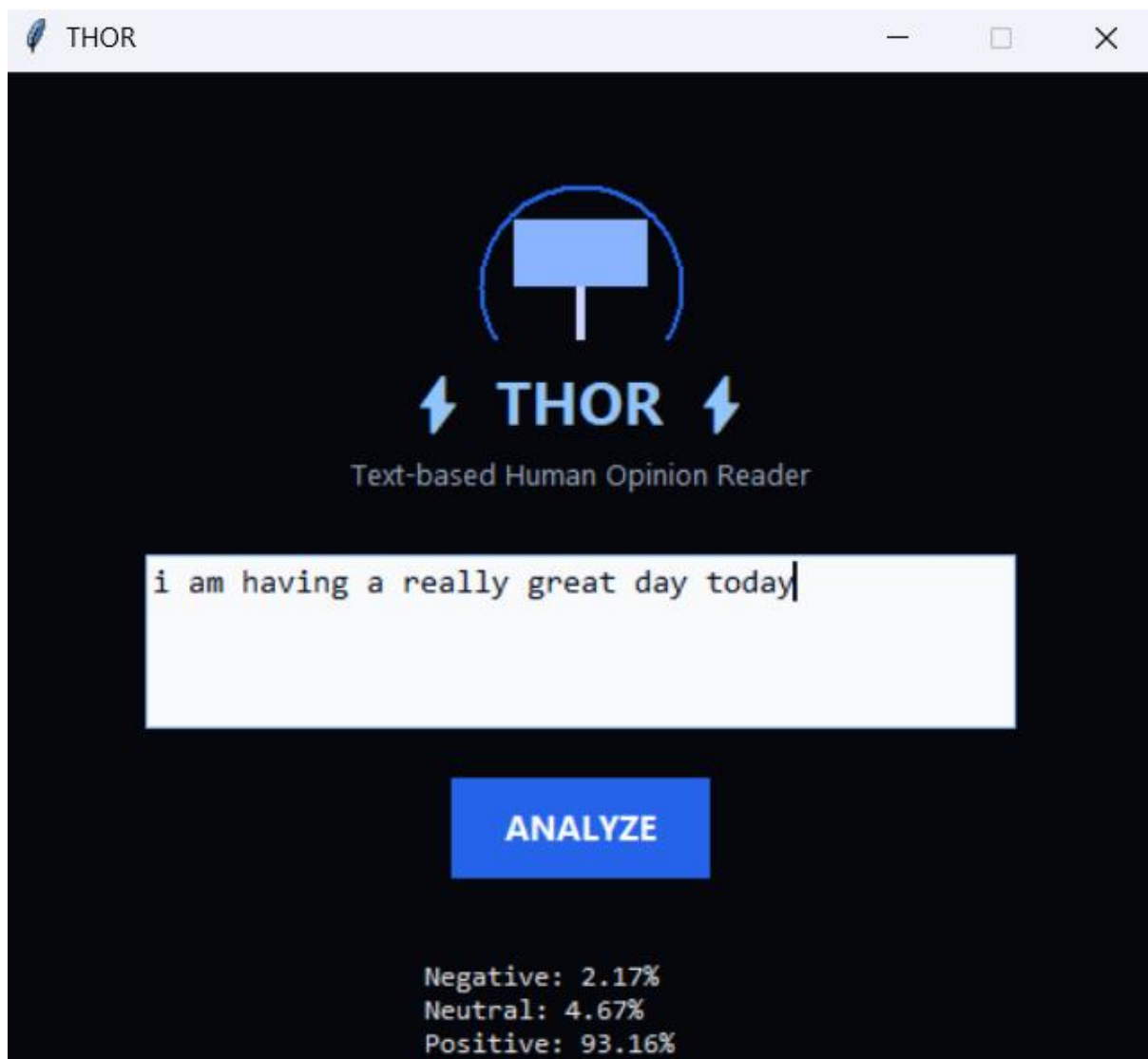
```

80         bg="█"#05070d",
81         fg="█"#94a3b8"
82     )
83     subtitle.pack(pady=(0, 15))
84
85     input_box = tk.Text(
86         root,
87         height=4,
88         width=48,
89         font=("Consolas", 11),
90         bg="█"#f8fafc",           # soft white
91         fg="█"#020617",          # dark text
92         insertbackground="█"#020617",
93         wrap="word",
94         relief="flat",
95         highlightthickness=1,
96         highlightbackground="█"#c7d2fe",
97         highlightcolor="█"#93c5fd"
98     )
99     input_box.pack(pady=10)
100
101
102     analyze_btn = tk.Button(
103         root,
104         text="ANALYZE",

```

```
105     command=analyze_text,
106     font=("Segoe UI", 11, "bold"),
107     bg=█ "#2563eb",
108     fg="white",
109     activebackground=█ "#1d4ed8",
110     relief="flat",
111     padx=18,
112     pady=6
113 )
114 analyze_btn.pack(pady=12)
115
116 ∨ result_label = tk.Label(
117     root,
118     text="",
119     font=("Consolas", 10),
120     bg=█ "#05070d",
121     fg=█ "#e5e7eb",
122     justify="left"
123 )
124 result_label.pack(pady=10)
125
126 root.mainloop()
127
```

## OUTPUT:



## 5. Using a Pre-Trained Model

Although there exist more powerful pre-trained models online that can detect sentiment in just few lines of code, I intentionally chose to:

- Go through the entire process
- Understand the basics of machine learning
- Build a simple model on my own to understand its power

Below is an approach where I used a pre-trained model:

## SOURCE CODE:

```

1  import mediapipe as mp
2  from mediapipe.tasks import python
3  from mediapipe.tasks.python import text
4  import os
5
6  MODEL_URL = (
7      "https://storage.googleapis.com/mediapipe-models/"
8      "text_classifier/bert_classifier/float32/1/bert_classifier.tflite"
9  )
10 MODEL_PATH = "sentiment_model.tflite"
11
12
13 base_options = python.BaseOptions(model_asset_path=MODEL_PATH)
14 options = text.TextClassifierOptions(
15     base_options=base_options,
16     max_results=2
17 )
18
19 classifier = text.TextClassifier.create_from_options(options)
20
21
22 def analyze_sentiment(input_text: str):
23     result = classifier.classify(input_text)
24
25     positivity = 0.0
26     negativity = 0.0
27
28     for category in result.classifications[0].categories:
29         label = category.category_name.lower()
30         score = category.score * 100

```

```

31
32         if "positive" in label:
33             positivity = score
34         elif "negative" in label:
35             negativity = score
36
37     return {
38         "text": input_text,
39         "positivity_percent": round(positivity, 2),
40         "negativity_percent": round(negativity, 2)
41     }
42
43
44 text_input = "I absolutely love this product, it works perfectly!"
45 output = analyze_sentiment(text_input)
46
47 print("INPUT:", output["text"])
48 print("POSITIVE:", output["positivity_percent"], "%")
49 print("NEGATIVE:", output["negativity_percent"], "%")

```

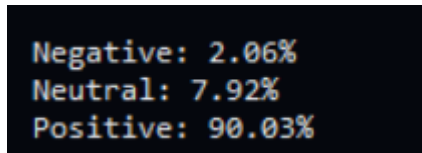
#### OUTPUT:

```

INPUT: I absolutely love this product, it works perfectly!
POSITIVE: 99.98 %
NEGATIVE: 0.02 %

```

And this is what my model predicted for the same prompt:



```
Negative: 2.06%  
Neutral: 7.92%  
Positive: 90.03%
```

### **Ideas to Implement**

Due to time constraints, I could not implement all my ideas for making a near-human-level prediction model.

Some future ideas include:

- Using OpenCV and DeepFace to predict the user's facial expression
- Using SpeechBrain to analyze tone of speech
- Combining text, facial expression, and tone to identify human emotions

With sufficient data, this system could:

- Interact like AI models such as ChatGPT or Gemini
- Reduce the gap between AI and human interaction ; )

## **Write your understanding of the project:**

Through this project, I gained a clear understanding of how text sentiment analysis works, starting from basic logic-based methods to advanced machine learning models. I learned the importance of data quality, feature weighting, and model selection in achieving accurate predictions. Building each model step-by-step helped me understand why advanced algorithms perform better and what challenges they face. Overall, this project strengthened my foundation in machine learning concepts and gave me confidence to build and improve intelligent systems in the future.

